

Proyecto Final

Paralelización del entrenamiento de múltiples MLP con distintas semillas

Computación Paralela y Distribuida (CS4052) – 2026-I

Kalos Lazo Aaron Navarro Fernando Usurin

Participación: 100 % cada uno (contribución equitativa)

Profesor: José Fiestas

Julio de 2026

1 Introducción

El proyecto consiste en paralelizar el entrenamiento de un conjunto de perceptrones multicapa (MLP) entrenados con distintas semillas aleatorias, conservando el de mayor exactitud sobre el conjunto de prueba. Dado que la inicialización de pesos depende de la semilla del generador pseudoaleatorio, una práctica robusta es entrenar varios modelos y conservar el mejor [4, 5].

Este procedimiento es **embarazosamente paralelo**: cada entrenamiento es independiente y los procesos solo se sincronizan al final para comparar exactitudes y seleccionar el mejor [2]. Es por ello un caso ideal para un modelo PRAM de **memoria compartida**.

Solución planteada. A partir del Algoritmo 1 (entrenamiento secuencial de múltiples MLP) proponemos:

1. Un algoritmo PRAM sobre el modelo CREW que reparte el conjunto de semillas $S = \{0, \dots, p_s - 1\}$ entre p procesadores y aplica una reducción final *argmáx* en árbol.
2. Una implementación en Python con `scikit-learn` [8, 9], paralelizada con `multiprocessing` (análogo a OpenMP [6, 7]).
3. Una caracterización empírica de tiempo, speedup, eficiencia y GFLOP/s para $p \in \{1, 2, 4, 8, 16, 32\}$ y $n \in \{5\,000, \dots, 40\,000\}$, con superposición de la curva teórica y análisis de escalabilidad (strong scaling bajo Amdahl [10, 12] y weak scaling bajo Gustafson [11]).

2 Método

2.1 Algoritmo secuencial de referencia

El Algoritmo 1 entrena p_s modelos MLP, uno por cada semilla, y conserva el de mayor exactitud.

Algorithm 1 Entrenamiento secuencial de múltiples MLP con distintas semillas

Require: Datos (X, y) , semillas $S = \{0, \dots, p_s - 1\}$, hiperparámetros $(\alpha, \eta, h, T_{\max})$

Ensure: Modelo M^* con mayor exactitud

- 1: Dividir (X, y) en train/test ▷ una sola vez
 - 2: $A_{\text{best}} \leftarrow -\infty$; $M^* \leftarrow \emptyset$
 - 3: **for all** $s \in S$ **do**
 - 4: inicializar RNG con semilla s ; construir y entrenar M_s
 - 5: $A_s \leftarrow \text{accuracy}(M_s, X_{\text{test}}, y_{\text{test}})$
 - 6: **if** $A_s > A_{\text{best}}$ **then** $A_{\text{best}} \leftarrow A_s$; $M^* \leftarrow M_s$
 - 7: **end if**
 - 8: **end for**
 - 9: **return** M^*
-

2.2 Algoritmo PRAM propuesto (CREW, memoria compartida)

Asumimos un PRAM **CREW** con p procesadores que comparten memoria global: varios procesadores leen concurrentemente el dataset y los hiperparámetros, pero las escrituras se serializan mediante una reducción *argmáx* [1]. Cada P_k trabaja sobre el subconjunto de semillas S_k , con $|S_k| \approx \lceil p_s / p \rceil$ (reparto balanceado). Conviene precisar una cuestión de modelo: en el informe parcial, la lectura de datos se denominó “Broadcast”, lo cual es impropio de la memoria compartida —allí los datos residen en memoria global y cada procesador simplemente los *lee* de forma concurrente (acceso CREW), sin necesidad de difundirlos—. Por ello se ha renombrado a **lectura concurrente** y, atendiendo la observación del docente, no se contabiliza como un paso paralelo independiente.

Algorithm 2 PRAM-CREW: entrenamiento de múltiples MLP (memoria compartida)

Require: (X, y) , $S = \{0, \dots, p_s - 1\}$, p procesadores, hiperparámetros

Ensure: M^* con mayor exactitud

- /* Paso 1 – Split (secuencial). NO se mide */
- 1: Dividir (X, y) ; **dejar** todo en memoria compartida
 - /* Paso 2 – Entrenamiento local (**pardo**) */
 - 2: **for all** $k = 0, \dots, p - 1$ **pardo do** ▷ $W_2 = O(p_s \text{ E n d } h)$, $P_2 = p$
 - 3: P_k **lee** de memoria compartida $(X_{\text{train}}, \dots)$ ▷ CREW
 - 4: entrenar las MLP de S_k ; guardar mejor $(A_{\text{best}}^{(k)}, M^{(k)})$ local
 - 5: **end for**
 - /* Pasos 3..2 + $\log_2 p$ – Reducción *argmáx* en árbol */
 - 6: **for** stride = 1, 2, 4, $\dots$, $p/2$ **do** ▷ stride $\leftarrow 2 \cdot$ stride cada ronda
 - 7: **for all** $k = 0, 2 \cdot \text{stride}, 4 \cdot \text{stride}, \dots$ **pardo do**
 - 8: **if** $A_{\text{best}}^{(k+\text{stride})} > A_{\text{best}}^{(k)}$ **then** $A_{\text{best}}^{(k)} \leftarrow A_{\text{best}}^{(k+\text{stride})}$; $M^{(k)} \leftarrow M^{(k+\text{stride})}$
 - 9: **end if**
 - 10: **end for**
 - 11: **end for**
 - 12: **return** $M^* = M^{(0)}$
-

Nota de implementación. La reducción *argmáx* en árbol de $\log_2 p$ rondas es el *modelo* PRAM que justifica el término $\log_2 p$ del costo (Sección 2.4); en la implementación, la reducción se realiza con un max serial sobre los p_s resultados durante la fase *join* del esquema *fork-join*. Ambas variantes

producen el mismo M^* y su diferencia es despreciable en la práctica, pues la reducción mide $\leq 3,6\%$ del tiempo total (Sección 3).

2.3 Justificación del uso de memoria compartida

1. **Comunicación mínima.** Es un esquema embarazosamente paralelo: la única interacción es la reducción argmáx final (árbol de $\log_2 p$ rondas). No se aprovecha nada del paso de mensajes.
2. **Datos compartidos y de solo lectura.** Cada MLP se entrena con todo el dataset. En memoria compartida basta con que los p procesadores *lean* las mismas referencias (herencia por *fork*); en memoria distribuida habría que copiar el dataset a cada proceso (memoria $\times p$). La memoria compartida es aquí más eficiente.
3. **Modelo de programación.** Se usa *multiprocessing* de Python como análogo de OpenMP [6, 7]: patrón *fork-join* donde los trabajadores heredan la memoria global y se sincronizan al recolectar resultados. El GIL de Python impide hilos reales, de ahí el uso de procesos.

2.4 Métricas y modelo de costo

A partir del Algoritmo 2 se obtiene el modelo de costo que permite, por un lado, predecir el comportamiento esperado del paralelo y, por otro, contrastarlo con las mediciones de la Sección 3. La descomposición paso a paso (paradigma *Work-Time*) asocia a cada etapa su trabajo W_i y el número de procesos activos P_i :

Paso	Descripción	W_i	P_i
1	Split y carga en memoria compartida	$O(n)$	1
2	Entrenamiento de $\lceil p_s/p \rceil$ MLP (pardo)	$O(p_s E n d h)$	p
3...	Reducción argmáx ($\log_2 p$ rondas)	$O(p)$	$p/2, \dots, 1$
final	Escritura de M^*	$O(1)$	1

El entrenamiento de los p_s modelos domina el trabajo total, $W = O(p_s E n d h)$, que coincide con el tiempo secuencial T_s : la paralelización *no añade trabajo*, únicamente lo redistribuye entre procesadores. El camino crítico (span) queda fijado por *un único* entrenamiento, $T_\infty = O(E n d h)$, y define el límite inferior del tiempo, por muchos procesadores que se asignen. Combinando ambas cotas mediante el Teorema de Brent se obtiene el modelo de costo

$$T_p \approx a \frac{p_s}{p} E n d h + b \log_2 p, \quad (1)$$

donde a es una **constante empírica** que traduce la complejidad estructural $E n d h$ a segundos (refleja la velocidad de la máquina) y b un **overhead empírico por ronda** que agrupa la sincronización *fork-join* y la contención de memoria; conviene subrayar que b *no* es la latencia de la reducción argmáx propiamente dicha, cuyo peso medido es $\leq 3,6\%$ de T_{total} (Sección 3). Ambas se estiman por mínimos cuadrados en la Sección 3.5. Conviene recalcar que a y $E d h$ son factores de naturaleza distinta: $E d h$ es la complejidad del MLP (fija), mientras que a depende del hardware.

De (1) se desprende que el speedup tiende a $S(p) \rightarrow p$ cuando el término de cómputo domina, comportamiento que la experimentación deberá confirmar. Asimismo, el costo $C_p = T_p \cdot p$ resulta $O(W)$ mientras $p \leq W/T_\infty = O(p_s)$; como todos los experimentos cumplen $p \leq p_s = 32$, el reparto de semillas aprovecha los procesadores sin dejarlos ociosos, esto es, el algoritmo es WT-óptimo.

Finalmente, el costo en operaciones de un MLP ($\text{FLOPs} = 4 E h (d + c) n$, forward+backward) habilita el reporte del rendimiento en GFLOP/s, métrica con la que se caracteriza además la saturación del hardware.

3 Resultados y análisis

3.1 Configuración experimental

Plataforma: cluster HPC **Khipu** de UTEC, nodo **n003** (Intel Xeon Gold 6130, 32 núcleos lógicos ≈ 16 físicos con hyper-threading, 224 GB DDR4), gestionado con Slurm. **Software:** Python 3.11, scikit-learn 1.9, numpy. **Hiperparámetros:** $p_s = 32$ semillas, $d = 32$ features, $h = 128$ neuronas, $c = 10$ clases, $E = 100$ épocas, $\alpha = 10^{-4}$, $\eta = 10^{-3}$. El BLAS se limita a 1 hilo por proceso ($\text{OMP_NUM_THREADS}=1$) para que la única paralelización sea la nuestra. Cada punto (n, p) se mide con 2 repeticiones; se reporta el **worker crítico** (máximo entre procesos), sin incluir la lectura ni el split. El código es determinista: la versión paralela reproduce exactamente las 32 exactitudes y el mismo modelo ganador que la secuencial.

3.2 Tiempos de ejecución

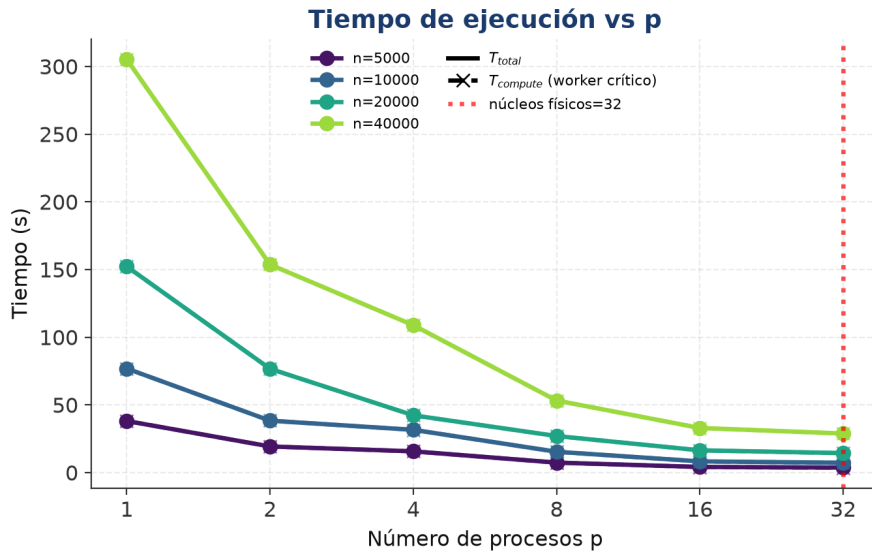


Figura 1: Tiempo vs p (Khipu, 32 núcleos). La reducción T_{reduce} es $\leq 3,6\%$ (y $< 1\%$ para n grande): el cuello no es la comunicación.

El tiempo cae con p sin saturarse prematuramente. La reducción T_{reduce} es $\leq 3,6\%$ de T_{total} (y $< 1\%$ para n grande), confirmando el carácter embarazosamente paralelo.

3.3 Speedup

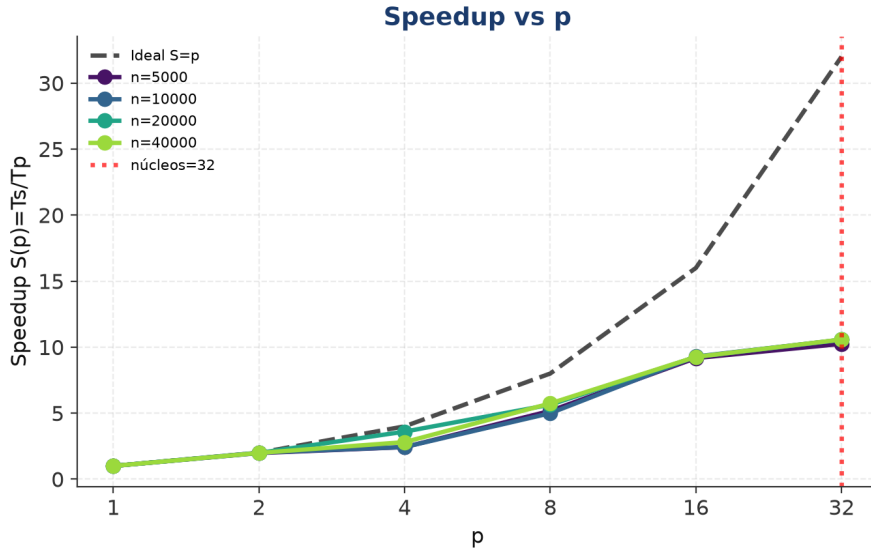


Figura 2: Speedup $S(p) = T_s/T_p$ vs p frente a la cota ideal $S = p$.

El modelo de costo de la Sección 2.4 anticipa un speedup cercano a p mientras el cómputo domine sobre la reducción. La Figura confirma esa predicción en la región baja: $S(p)$ sigue la diagonal ideal hasta $p \approx 2$ ($E \approx 0,99$) y conserva una eficiencia razonable ($E \approx 0,6-0,9$ según n) hasta $p \approx 8$, lo que valida tanto el reparto de semillas como la hipótesis de trabajo dominante. Se observa una anomalía puntual en $p = 4$ —con eficiencia incluso inferior a la de $p = 8$ para los n pequeños y varianza alta entre repeticiones—, atribuible a efectos de turbo-boost/NUMA con pocos procesos activos. Más allá de $p \approx 8$ el crecimiento se atenúa y se estabiliza en torno a $S \approx 10,6$ para $p = 32$. Este techo, muy por debajo de los 32 núcleos lógicos del nodo, no obedece a la comunicación ($T_{\text{reduce}} \leq 3,6\%$) sino a dos factores del hardware: el **hyper-threading** presenta 32 núcleos lógicos que en realidad son ≈ 16 físicos, acotando el speedup real al intervalo $[10, 16]$; y la **contención del ancho de banda de memoria**, propia de una carga *memory-bound*, desacelera cada MLP desde ~ 9.6 s (aislado) hasta ~ 28.9 s cuando 32 procesos compiten por la memoria. El siguiente análisis de eficiencia permite cuantificar cuánto de ese techo se traduce en aprovechamiento real.

3.4 Eficiencia

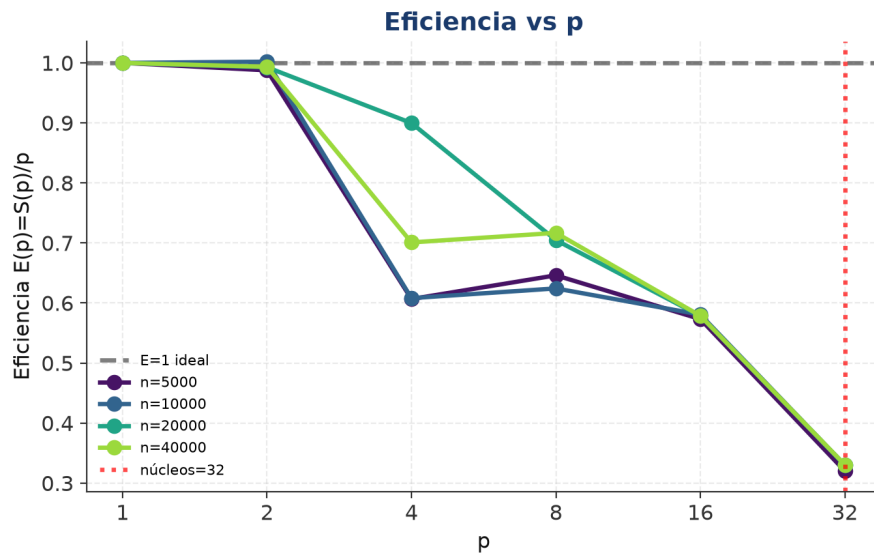


Figura 3: Eficiencia $E(p) = S(p)/p$ vs p .

La eficiencia es la contraparte informativa del speedup: mide cuánto aporta cada procesador adicional. Bajo *strong scaling* decrece con p —de forma no estrictamente monótona, por el bache en $p = 4$ ya señalado—, pasando de ~ 0.99 en $p = 2$ a ~ 0.33 en $p = 32$, reflejando los mismos límites identificados arriba (hyper-threading y contención). Esta degradación es mucho más modesta bajo *weak scaling* (Sección 3.8), donde la carga por procesador se conserva: aunque tampoco es perfecta —la naturaleza *memory-bound* del MLP se hace sentir al crecer el problema—, la pérdida de eficiencia es claramente inferior. En consecuencia, la eficiencia **no es constante** en escalabilidad fuerte, y se preserva bastante mejor en escalabilidad débil. Resta verificar si el modelo teórico reproduce este comportamiento.

3.5 Validación frente a la complejidad teórica

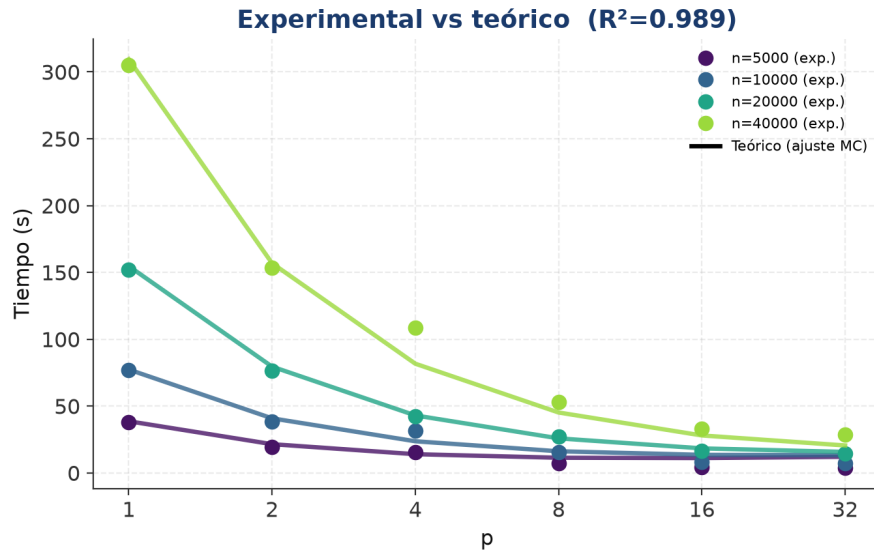


Figura 4: Tiempo experimental (puntos) frente a la curva teórica de la Ec. (1) ajustada por mínimos cuadrados no-negativos (líneas).

Para contrastar el modelo de costo con la realidad se ajustó la Ec. (1) por mínimos cuadrados no-negativos sobre los 48 puntos experimentales. El ajuste arroja $R^2 = 0.989$, con $a \approx 5,9 \times 10^{-10}$ s y $b = 2.18$ s. Como muestra la Figura, el término dominante $a(p_s/p) \log_2 p$ reproduce fielmente la región lineal, mientras que el término $b \log_2 p$ (con $b \approx 2,18$ s) no representa la reducción $\arg\max$ —cuyo peso medido es $\leq 3,6\%$ de T_{total} — sino que absorbe empíricamente el overhead de sincronización y la contención de memoria a alto p que el término lineal no captura. El elevado R^2 permite concluir que la expresión teórica —que, a diferencia del parcial, **incluye el término $\log_2 p$** de la reducción— constituye una descripción correcta del costo del algoritmo.

3.6 Rendimiento (GFLOP/s)

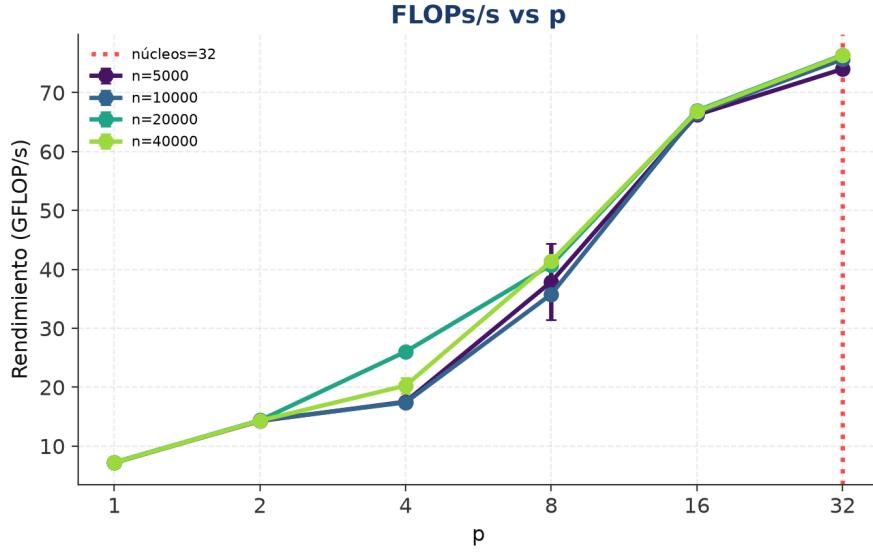


Figura 5: Rendimiento (GFLOP/s) vs p .

El rendimiento complementa la lectura del speedup desde la óptica de las operaciones: crece de forma sostenida hasta ~ 67 GFLOP/s en $p = 16$ y ~ 76 GFLOP/s en $p = 32$. Que la curva se sature al mismo nivel que el speedup, y no siga escalando con p , confirma que el cuello final no son los núcleos sino el ancho de banda de memoria compartido. Los dos análisis siguientes caracterizan ambos regímenes de escalabilidad para cerrar esta interpretación.

3.7 Escalabilidad fuerte

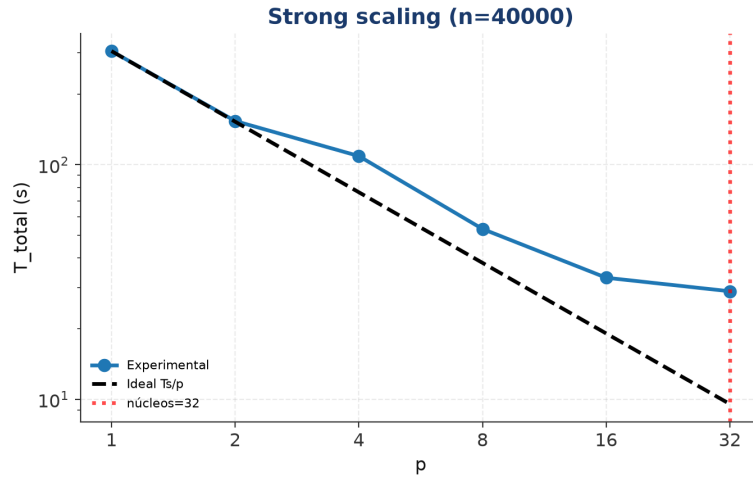


Figura 6: Strong scaling para $n = 40000$ (log-log): la pendiente sigue $1/p$ hasta $p \approx 8$ y luego se aplan.

Con el tamaño del problema fijo, la complejidad empírica sigue $T_p \propto 1/p$ en la región lineal, confirmando el costo $\Theta(En d h)$ por procesador derivado en la Sección 2.4. El aplanamiento posterior es la signature del límite por ancho de banda ya identificado. Este régimen, sin embargo,

es el más exigente; conviene examinar el comportamiento cuando el problema crece junto con p .

3.8 Escalabilidad débil

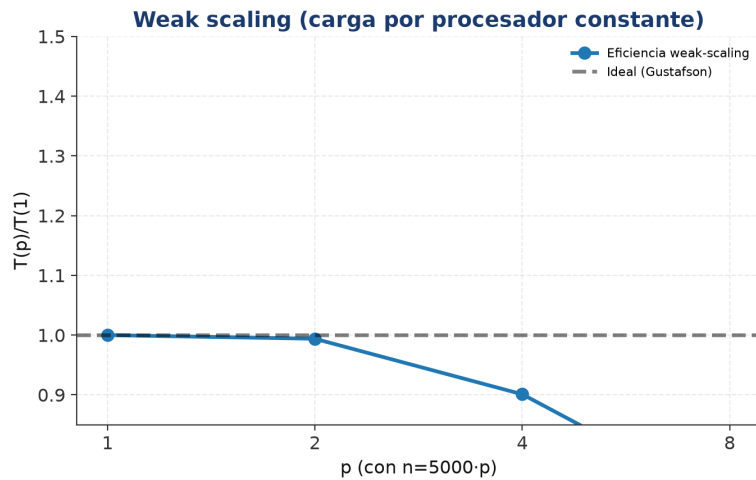


Figura 7: Weak scaling con $n = 5000 \cdot p$: la cercanía a 1 indica escalabilidad ideal bajo Gustafson.

Bajo weak scaling (n crece proporcionalmente con p) el cociente $T(p)/T(1)$ se mantiene cercano a la unidad en los primeros pasos y luego crece moderadamente, hasta ≈ 1.4 en $p = 8$: aunque cada procesador conserva su carga nominal, el mayor tamaño total del problema intensifica la contención de memoria. Esta degradación es, no obstante, claramente inferior a la del strong scaling, lo que confirma que el algoritmo aprovecha mejor los recursos cuando el problema crece junto con ellos, en línea con la ley de Gustafson.

3.9 Mejoras propuestas

El análisis anterior sugiere varias líneas de mejora. La contención de memoria a alto p podría aliviarse con una **asignación dinámica** de semillas que balancee MLPs de duración heterogénea. El techo impuesto por Python invita a un **backend nativo OpenMP/C++** que elimine el overhead de proceso, y a explotar un **segundo nivel de paralelismo dentro del MLP** (BLAS multihilo o GPU) debidamente coordinado con el de semillas. Finalmente, escalar a un nodo de **128 núcleos** permitiría verificar el comportamiento más allá del rango explorado.

3.10 Tabla resumen

n	p	T_{total} (s)	Speedup	Eficiencia	GFLOP/s	best-acc
5000	1	38.11	1.00	1.000	7.2	0.765
5000	2	19.30	1.98	0.988	14.3	0.765
5000	4	15.71	2.43	0.607	17.5	0.765
5000	8	7.37	5.17	0.646	37.9	0.765
5000	16	4.16	9.17	0.573	66.2	0.765
5000	32	3.72	10.25	0.320	74.0	0.765
10000	1	76.85	1.00	1.000	7.2	0.798
10000	2	38.35	2.00	1.002	14.4	0.798
10000	4	31.60	2.43	0.608	17.4	0.798
10000	8	15.39	4.99	0.624	35.8	0.798
10000	16	8.27	9.29	0.581	66.5	0.798
10000	32	7.28	10.56	0.330	75.7	0.798
20000	1	152.25	1.00	1.000	7.2	0.856
20000	2	76.68	1.99	0.993	14.4	0.856
20000	4	42.31	3.60	0.900	26.0	0.856
20000	8	27.01	5.64	0.705	40.8	0.856
20000	16	16.45	9.26	0.578	66.9	0.856
20000	32	14.41	10.56	0.330	76.4	0.856
40000	1	305.26	1.00	1.000	7.2	0.868
40000	2	153.64	1.99	0.993	14.3	0.868
40000	4	108.88	2.80	0.701	20.3	0.868
40000	8	53.24	5.73	0.717	41.4	0.868
40000	16	32.97	9.26	0.579	66.8	0.868
40000	32	28.86	10.58	0.331	76.3	0.868

4 Bibliografía

Los fundamentos de redes neuronales y retropropagación se sustentan en [3, 4, 5], que justifican el costo $\Theta(E n d h)$. La implementación usa `scikit-learn` [8, 9]. El modelo de memoria compartida se apoya en [6, 7]; el carácter embarazosamente paralelo y la taxonomía del paralelismo en entrenamiento, en [1, 2]. Las leyes de escalabilidad en [10, 11, 12]. El uso de IA generativa (Claude, Anthropic) fue como co-piloto para redacción y verificación; toda derivación, decisión de diseño y código fue validada manualmente por los integrantes.

Referencias

- [1] T. Ben-Nun and T. Hoefler, “Demystifying parallel and distributed deep learning: An in-depth concurrency analysis,” *ACM Computing Surveys*, vol. 52, no. 4, art. 65, 2019. doi:10.1145/3320060.
- [2] F. C. Farias, T. B. Ludermir and C. J. A. Bastos-Filho, “Embarrassingly parallel independent training of multi-layer perceptrons with heterogeneous architectures,” *AI (MDPI)*, vol. 4, no. 1, pp. 16–27, 2023. doi:10.3390/ai4010002.
- [3] D. E. Rumelhart, G. E. Hinton and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, no. 6088, pp. 533–536, 1986. doi:10.1038/323533a0.
- [4] Y. LeCun, Y. Bengio and G. Hinton, “Deep learning,” *Nature*, vol. 521, no. 7553, pp. 436–444, 2015. doi:10.1038/nature14539.

- [5] I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*. Cambridge, MA: MIT Press, 2016. www.deeplearningbook.org.
- [6] L. Dagum and R. Menon, "OpenMP: An industry-standard API for shared-memory programming," *IEEE Computational Science & Engineering*, vol. 5, no. 1, pp. 46–55, 1998. doi:10.1109/99.660313.
- [7] B. Chapman, G. Jost and R. van der Pas, *Using OpenMP: Portable Shared Memory Parallel Programming*. Cambridge, MA: MIT Press, 2007.
- [8] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. jmlr.org/papers/v12/pedregosa11a.
- [9] L. Buitinck *et al.*, "API design for machine learning software: Experiences from the scikit-learn project," in *ECML PKDD Workshop: Languages for Data Mining and Machine Learning*, 2013, pp. 108–122.
- [10] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *Proc. AFIPS Spring Joint Computer Conf.*, vol. 30, 1967, pp. 483–485. doi:10.1145/1465482.1465560.
- [11] J. L. Gustafson, "Reevaluating Amdahl's law," *Communications of the ACM*, vol. 31, no. 5, pp. 532–533, 1988. doi:10.1145/42411.42415.
- [12] M. D. Hill and M. R. Marty, "Amdahl's law in the multicore era," *IEEE Computer*, vol. 41, no. 7, pp. 33–38, 2008. doi:10.1109/MC.2008.209.

El trabajo computacional de este proyecto fue apoyado por los recursos del cluster HPC **Khipu** (web.khipu.utec.edu.pe) de la Universidad de Ingeniería y Tecnología (UTEC).